Reviewer Pack — Minimal Number Field Trace Correlation with Circuit Monotone…

Entry b8c708298cbd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 19:56:30 UTC

Minimal Number Field Trace Correlation with Circuit Monotone Width

Entry ID: `b8c708298cbd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 19:56:30 UTC

1. Conjecture

Field A (mathematical branch): Number Field Theory **Field B** (complexity object): Boolean Circuits

Statement:

> For every satisfiable CNF formula φ, the minimal number field trace (mnt(φ)) of its associated algebraic closure is linearly correlated with its circuit monotone width w(φ), such that mnt(φ) = Θ(w(φ)).

Rationale (proposer's reasoning):

> Number field theory provides a rich structure for studying algebraic extensions of fields, which can potentially reveal non-trivial properties of the circuits computing the Boolean functions associated with CNF formulas. The minimal number field trace captures the complexity of the smallest field containing all solutions to a given polynomial equation, and this invariant could be linked to the structural complexity of the circuit representing the satisfiability problem.

Taxonomy category: `TROPICAL_FOURIER_ANALYSIS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `539fc2fe829b8a8a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF formula φ with n variables (n ≤ 40), if the correlation coefficient r^2 between the minimal number field trace mnt(φ) and circuit monotone width w(φ) is >= 0.8 for all seeds, then support the conjecture.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'number field theory' AND 'Boolean circuits' AND 'trace correlation'` - `'mnt(φ)' AND 'circuit monotone width' AND CNF` - `'minimal number field trace' related to 'circuit monotonicity'`

Top relevant hits considered: - [http://arxiv.org/abs/1409.5893v2] Fast evaluation of far-field signals for time-domain wave propagation - [http://arxiv.org/abs/physics/0604089v3] Space-time transformation properties of inter-charge forces and dipole radiation: Breakdown of the classical field conce - [http://arxiv.org/abs/0704.1574v3] Retarded electric and magnetic fields of a moving charge: Feynman's derivation of Liénard-Wiechert potentials revisited - [http://arxiv.org/abs/2107.12643v2] On φ-w-Flat modules and Their Homological Dimensions - [http://arxiv.org/abs/1806.06693v3] Higher-order field theories: φ^6, φ^8 and beyond - [http://arxiv.org/abs/1610.01843v3] NNLO QCD corrections for Drell-Yan p_T^Z and φ^* observables at the LHC - [http://arxiv.org/abs/2402.07935v3] On the Frobenius fields of abelian varieties over number fields - [http://arxiv.org/abs/2604.22734v1] Radiation outer boundary conditions and near-to-far field signal transformations for the Bardeen-Press equation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = -matrix[j][i] / matrix[i][i]
            for k in range(n):
                matrix[j][k] += factor * matrix[i][k]

    # Back substitution
    x = [0] * n
```

```
    for i in range(n-1, -1, -1):
        x[i] = matrix[i][-1]
        for j in range(i+1, n):
            x[i] -= matrix[i][j] * x[j]
        x[i] /= matrix[i][i]

    return x

def random_cnf(n):
    clauses = []
    for _ in range(2**n // 3):  # Ensure satisfiability
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        if len(set(abs(lit) for lit in clause)) == n:
            clauses.append(clause)
    return clauses

def cnf_to_circuit(cnf):
    # Simplified circuit construction using a lookup table
    n = max(abs(lit) for clause in cnf for lit in clause)
    truth_table = [[0] * (2**n) for _ in range(n)]

    def evaluate(variable, assignment):
        if variable > 1:
            return evaluate(-variable // 2, assignment) and evaluate(variable % 2, assignment)
        elif variable == 1:
            return assignment[0]
        else:
            return not assignment[0]

    for i in range(n):
        for j in range(2**n):
            truth_table[i][j] = evaluate(i + 1, [bool(j & (1 << k)) for k in range(n)])

    circuit = []
    for clause in cnf:
        gate = []
        for lit in clause:
            if lit > 0:
                gate.append(truth_table[lit - 1])
            else:
                gate.append([not x for x in truth_table[-lit - 1]])
        circuit.append(gate)

    return circuit

def monotone_width(circuit):
    n = len(circuit)
    dp = [[0] * (2**n) for _ in range(n)]

    def evaluate_gate(gate, assignment):
        return any(all(assignment[j] for j in gate[i]) for i in range(len(gate)))

    for i in range(n):
        for j in range(2**n):
            dp[i][j] = max(dp[i-1][k] if k & (1 << i) == 0 else dp[i-1][k] + evaluate_gate(circuit
```

```
    return max(max(row) for row in dp)

def minimal_number_field_trace(cnf):
    # Simplified computation using a lookup table
    n = max(abs(lit) for clause in cnf for lit in clause)
    truth_table = [[0] * (2**n) for _ in range(n)]

    def evaluate(variable, assignment):
        if variable > 1:
            return evaluate(-variable // 2, assignment) and evaluate(variable % 2, assignment)
        elif variable == 1:
            return assignment[0]
        else:
            return not assignment[0]

    for i in range(n):
        for j in range(2**n):
            truth_table[i][j] = evaluate(i + 1, [bool(j & (1 << k)) for k in range(n)])

    trace = sum(truth_table[i][i] for i in range(n))
    return trace

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    mnt_phi_list = []
    circuit_phi_list = []

    for n in n_values:
        cnf = random_cnf(n)
        mnt_phi = minimal_number_field_trace(cnf)
        circuit = cnf_to_circuit(cnf)
        circuit_phi = monotone_width(circuit)

        if mnt_phi is not None and circuit_phi is not None:
            mnt_phi_list.append(mnt_phi)
            circuit_phi_list.append(circuit_phi)

    instances_tested = len(mnt_phi_list)
    n_max = max(n_values)

    if instances_tested == 0:
        return {
            "metric_name": "mnt_phi vs circuit_phi",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    mean_metric_value = sum(mnt_phi_list) / instances_tested
    mean_circuit_phi = sum(circuit_phi_list) / instances_tested

    total_metric_value = sum(mnt_phi_list)
```

```
    if instances_tested == 1:
        r_squared = 1.0
    else:
        numerator = instances_tested * total_metric_value**2 - sum(mnt_phi**2 for mnt_phi in mnt_p
        denominator = (instances_tested - 1) * sum((mnt_phi - mean_metric_value)**2 for mnt_phi in
        r_squared = numerator / denominator if denominator != 0 else 0

    return {
        "metric_name": "mnt_phi vs circuit_phi",
        "metric_value": r_squared,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": r_squared >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_r_squared = sum(result["metric_value"] for result in results if result["metric_value"] is
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_r_squared} std=0.0 support_fraction={support_fraction
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"r_squared < 0.8\" first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fea509da.py", line 170, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fea509da.py", line 124, in run_trial
    circuit_phi = monotone_width(circuit)
                  ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fea509da.py", line 90, in monotone_widt
    dp[i][j] = max(dp[i-1][k] if k & (1 << i) == 0 else dp[i-1][k] + evaluate_gate(circuit[i], [bool(j &
                                  ^
NameError: name 'k' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17778
2	propose	ollama_remote	glm4:latest	0	0	30435
3	preregistration	ollama_remote	glm4:latest	0	0	10076
4	novelty	ollama_remote	glm4:latest	0	0	11763
5	novelty	ollama_remote	glm4:latest	0	0	21085
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13183
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16558
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	103937
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34899
10	judge	ollama_remote	glm4:latest	0	0	9627

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 269342 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b8c708298cbd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b8c708298cbd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b8c708298cbd.tar.gz` (if generated)